CoreVectorBlox IP Handbook

Introduction (Ask a Question)

CoreVectorBlox provides a flexible neural network accelerator. It uses an overlay approach, where one instantiation can run different networks without needing to be resynthesized.

This handbook provides details about the Microchip CoreVectorBlox environment and how to use it. This document is used by Microchip FPGA designers using Libero® System-on-Chip (SoC).

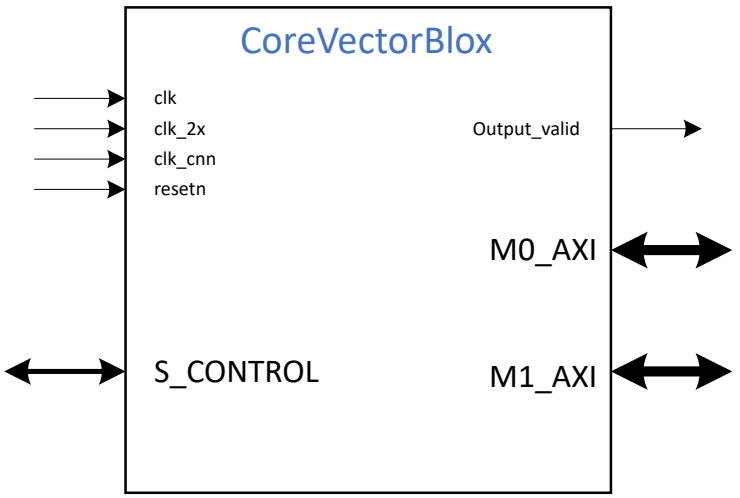
Table of Contents

1. Overview (Ask a Question)

CoreVectorBlox provides a flexible neural network accelerator. It uses an overlay approach, where one instantiation can run different networks without needing to be resynthesized. A software toolkit called VectorBlox Accelerator Software Development Kit (SDK) compiles a neural network description from a supported framework (for example, TensorFlow, Caffe and so on) into a Binary Large Object (BLOB) that is loaded into memory accessible by the CoreVectorBlox memory-mapped master. The CoreVectorBlox reads this BLOB and the network inputs from memory, processes the network, and places the result into an output buffer in memory. It can dynamically switch between multiple networks due to its overlay design. The overlay features a vector processor that can handle general vector layers and a convolutional accelerator, which further accelerates Convolutional Neural Networks (CNNs). CoreVectorBlox efficiently supports most convolutional neural networks available today, such as ResNet, MobileNet, YOLO and many more. Different configurations are available, allowing the user to scale the resource utilization to the required network performance.

The following figure shows the top-level interface.

Figure 1-1. CoreVectorBlox I/O Signal Diagram



1.1. Features (Ask a Question)

The following are the key features of CoreVectorBlox.

- Multiple size configurations to trade-off performance for resource utilization.
- Overlay design, which allows multiple networks to run on the same core and even switch dynamically.
- Two AXI4 memory masters with configurable width (64-bit to 256-bit) for data access.
- AXI4-Lite slave for control and status.
- Memory-based; reads inputs from and writes outputs to memory-mapped master.
- Internal vector processor, which can process general neural-network layers.
- CNN accelerator for convolutional layers.

1.2. Core Versions (Ask a Question)

This handbook applies to CoreVectorBlox v3.0. The release notes provided with the core list known discrepancies between this handbook and the core release associated with the release notes.

1.3. Supported Families (Ask a Question)

CoreVectorBlox supports the following devices:

- PolarFire®
- PolarFire® SoC

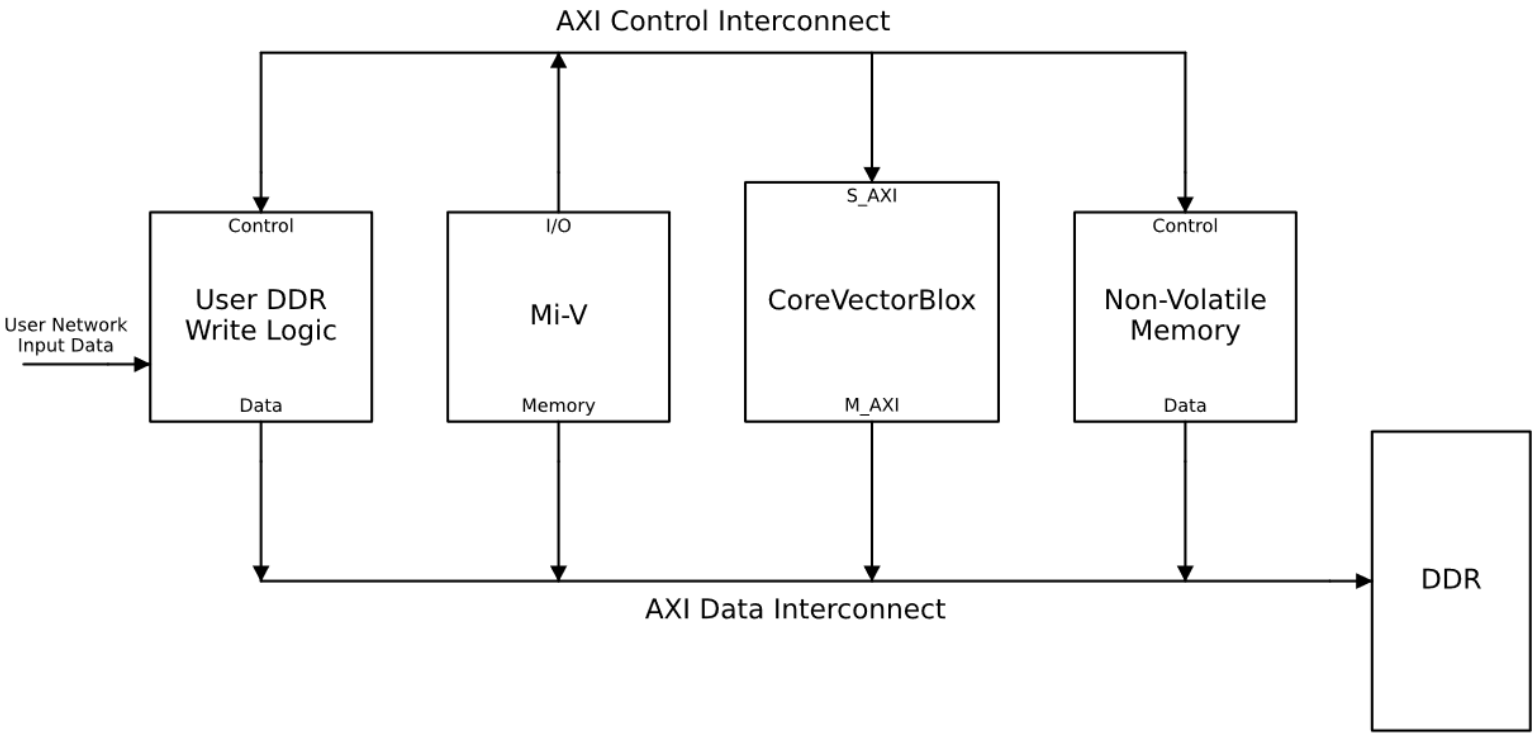
2. Functional Description (Ask a Question)

The following section shows the functional description of CoreVectorBlox.

2.1. System Level Overview (Ask a Question)

CoreVectorBlox processes neural networks residing in external memory. Its interfaces are an AXI4-Lite slave for setup and control and two AXI4 masters for instruction and data memory. The following figure shows an example system of CoreVectorBlox usage.

Figure 2-1. Example of System Level Block Diagram



The Mi-V soft processor controls the flow of data among components. At boot up, network BLOBs (processed by the SDK and stored in non-volatile memory) are copied from non-volatile memory to DDR memory. Incoming data (for example, video frames from an image sensor) is written into DDR memory by user logic under the control of the Mi-V. When new data is available, the Mi-V instructs CoreVectorBlox to begin processing. CoreVectorBlox reads the weights and layer types from the network BLOB and data from the network inputs from DDR memory, processes the network and then writes the results back to DDR memory. Finally, the Mi-V either directly reads the results or signals another module to consume them.

2.2. Memory Components (Ask a Question)

CoreVectorBlox requires the following components to be placed in memory accessible to its AXI4 master interface:

- Network BLOB(s)—BLOBs produced by the VectorBlox Accelerator SDK for each network to run.
- Network Inputs/Outputs—Network specific I/O for each run of a network.

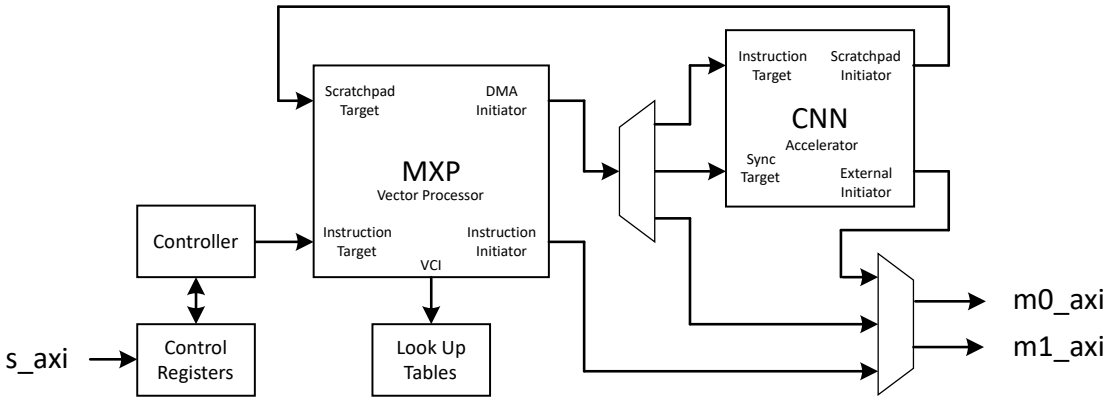
The network BLOBs are compiled by the user to target their own networks. Microchip provides examples and tutorials in the VectorBlox Accelerator SDK. A network BLOB contains information about the specific layer types in the network, as well as weights and buffer space for activations.

Each network has its own BLOB; multiple BLOBs can be present in memory at the same time and each time the CoreVectorBlox is started, it can target a different BLOB. The user loads the network BLOB(s) into external memory during bootup. The network inputs and outputs are specific to each network. For more information, see the VectorBlox™ Accelerator SDK.

2.3. Hardware Architecture (Ask a Question)

The following figure shows the primary blocks of CoreVectorBlox.

Figure 2-2. CoreVectorBlox Block Diagram



The following list describes the primary blocks of CoreVectorBlox:

- Control Registers—Control, status and error registers as well as addresses for BLOBs and I/O.
- Controller—Provides control instructions to the vector processor.
- MXP Vector Processor—Processes general neural network layers.
- CNN Accelerator—Processes convolutional network layers.

The control registers have an AXI4-Lite slave interface for external logic (for example, a Mi-V soft processor) to control the state of CoreVectorBlox. They are used to initialize and communicate with the controller. Once initialized, the controller sets status and error conditions in the control registers.

The controller issues execution instructions to the MXP and sets status and error registers based on the MXP operation.

The MXP Vector Processor is a soft vector processor, which performs data-parallel operations on long vectors of data. It uses an internal scratchpad as working memory and a variable-width DMA controller for transferring data between the scratchpad and external memory. The CoreVectorBlox size configuration (see Configuration Options) configures the number of parallel ALUs and scratchpad memory size of the MXP.

The CNN Accelerator is an array of Processing Elements (PEs) that consist of a multiply-accumulate unit and small accumulation RAM. Data are retrieved from external memory and then written to the MXP scratchpad or back to external memory. The PEs are laid out in a 2D grid, which takes advantage of the multiple levels of parallelism in many neural network layers. This achieves a higher level of performance than the MXP alone, especially in convolutional neural networks and achieves a higher degree of parallelism than the MXP, when processing the convolutional layers.

2.4. Configuration Options (Ask a Question)

CoreVectorBlox can be configured to support different compression as well as various sizes listed in the following table. Detailed resource usage, device utilization and benchmark along with power and memory bandwidth usage are available in the CoreVectorBlox SDK.

The following table lists the compression configuration details.

Table 2-1. Compression Configuration Details

Compression Configuration	Description
No Compression	This configuration has no acceleration for sparsity or pairing. This is the recommended configuration for use with models that are not trained with sparsity.
Compression	This configuration has special acceleration for sparsity and pairing in a structured format. For maximum acceleration, models must be trained with the VectorBlox Training Compression Library. This configuration also accelerate models trained with 2:4 structured sparsity.
Unstructured Compression	This configuration has special acceleration for sparsity and pairing in an unstructured format. Unstructured compression enables the highest acceleration for networks at the best performance/accuracy trade-off. For maximum acceleration, models must be trained with the VectorBlox Training Compression Library. This configuration also accelerate models trained with any structured or unstructured sparsity scheme.

The no compression configuration supports the following size configurations.

Table 2-2. Size Configuration—No Compression Configuration

Size Configuration	Vector Processor Width	Scratchpad Size	CNN Peak Ops (Operations per Clock Cycle)
V250	64-bit	32 kB	256
V500	128-bit	64 kB	512
V1000	256-bit	128 kB	1024

The compression configuration supports the following size configurations.

Table 2-3. Size Configurations—Compression

Size Configuration	Vector Processor Width	Scratchpad Size	CNN Equivalent Peak Ops (Operations per Clock Cycle)
V250	64-bit	32 kB	512
V500	128-bit	64 kB	1024
V1000	256-bit	128 kB	2048

The unstructured compression configuration is fixed to the following size.

Table 2-4. Size Configurations—Unstructured Compression

Size Configuration	Vector Processor Width	Scratchpad Size	CNN Equivalent Peak Ops (Operations per Clock Cycle)
Fixed	64-bit	32 kB	1792

3. Operation (Ask a Question)

The following section shows the operation of CoreVectorBlox.

3.1. Memory Map (Ask a Question)

The following table lists the control slave memory map. All registers are 32-bit in width. Register access is specified as a combination of readable (R), writable (W), write-to-set (WS) and write-to-clear (WC). Write-to-set (WS) bits are set to '1' by a write to the specified register that has a '1' in the WS bit, but can only be cleared by an internal logic (a write to the specified register with a '0' in the WS bit has no effect). Write-to-clear (WC) bits are cleared to '0' by a write to the specified register with a '1' in the WC bit (a write to the specified register with a '0' in the WC bit has no effect). Write-to-clear (WC) registers are cleared by any write to the specified register.

Table 3-1. Control Slave Memory Map

Address	Name	Description			Access
		Bit 0 = LSB	Function	Reset Value	
0x00	Control Register	0: Soft Reset	Write 1 to soft reset. All other control register bits are ignored when soft reset is activated. The error register is cleared on soft reset as well. Soft reset expects to fulfill memory transactions on all interfaces. If the modules connected to `S_AXI` or `MX_AXI` are placed into reset and may not have correctly fulfilled outstanding AXI transactions, then a hard reset (through the `resetn` pin) must be performed to ensure that the `S_AXI` and `MX_AXI` interfaces come up correctly.	1	R/W
		1: Start	Write 1 to set. Setting this bit causes the CoreVectorBlox to start processing a network. It begins fetching the network BLOB from the location specified in the Network Model Address register and decode the BLOB to determine how to proceed. The Network Model Address and Network I/O Address registers must be set before setting this bit. Once cleared, the hardware starts processing the current network (concurrently with setting the Running bit). While set, it is an error to Start again or to write to the Network Model Address or Network I/O Address registers.	0	R/WS
		2: Running	Set during processing (concurrently with clearing the Start bit). Cleared concurrently to setting the "Output Valid" bit.	0	R

Table 3-1. Control Slave Memory Map (continued)

Address	Name	Description			Access
		Bit 0 = LSB	Function	Reset Value	
0x00	Control Register	3 Output Valid	Write 1 to clear. Set once the network outputs are valid. It must be cleared once per network invocation. An invocation is started by writing the Start bit and is ended by clearing the Output Valid bit. The Output Valid bit is not cleared before writing the Start bit for the next network invocation, but until it is cleared, the subsequent network does not finish (the Running bit stays set). The Output Valid bit sets once and must be cleared once per setting of the Start bit. If the control slave does not clear this bit, subsequent network invocations will not finish. While this bit is clear, it is an error to write to this bit.	0	R/WC
0x00	Control Register	4: Error	Indicates the contents of the Error Register are valid and must be examined.	0	R
		31:5	Reserved	0	R
0x04	Error Register	Write any value to clear to zero, which also clears the Error bit of the Control Register. Errors will cause a soft reset, which is identical to setting the Soft Reset bit of the Control Register except that the Error bit and this register are not cleared. See Error Codes for more information.			R/WC
0x08	Reserved	—			—
0x10	Network Model Address	Address Pointing to Model BLOB. Must be aligned to an 8 byte boundary and be greater than or equal to 0x0020_0000 when setting the "Start" bit or an error will be raised. Writing while the Start bit is set raises an error.			R/W
0x18	Network I/O Address	Address pointing to the I/O data structure for the network. Must be aligned to an 8 byte boundary and be greater than or equal to 0x0020_0000 when setting the "Start" bit or an error will be raised. Writing while the Start bit is set raises an error.			R/W
0x28	Version	See the following table for version information.			R

Table 3-2. Version

Bits	Name	Function
7:0	Product ID	Reserved; Reads 0 for CoreVectorBlox.
15:8	Size Configuration	Size configuration: 0: V250 1: V500 2: V1000
19:16	Compression Configuration	0: No Compression 1: Compression 2: Unstructured Compression
27:20	Minor Version	Minor version number (0x00 for CoreVectorBlox 3.0)
31:28	Major Version	Major version number (0x03 for CoreVectorBlox 3.0)

Table 3-3. Additional Registers for Unstructured Compression

Register Address	Register Name	Description					Access
		Field_Name	Function	Start Bit	Num Bits	Default Value	
0x800	AP_CONTROL	0: ap_start	When writing "1" the engine start to work	0	1	0	RW
		1: ap_done	Engine set to "1" when it completes his work, also this bit is Clear on reset (AWS-Shell needs this)	1	1	0	R/COR
		2: ap_idle	Read engine status; when it is working it should be at "1" state	2	1	0	R
		3: ap_ready	—	3	1	0	R
		4: ap_continue	—	4	1	0	RW
		5: Reserved	—	5	2	0	R
		7: auto_restart	—	7	1	0	RW
		8: Reserved	—	8	24	0	R
0x804	AP_GIE	0: global_ienable	Global interrupt enable	0	1	0	RW
0x808	AP_IER	0: ip_ienable	IP interrupt enable	0	1	0	RW
0x80c	AP_ISR	0: ip_istatus	IP interrupt status	0	1	0	RW
0x810	AP_VER_ID	0: version_id	Version ID of the core	0	32	0	R
0x814	RESERVED	0: Reserved	—	0	32	0	R
0x818	AP_VER_TS	0: version_id_lsb	Version Timestamp of bitfile's creation	0	32	0	R
0x81c	RESERVED	0: Reserved	—	0	32	0	R
0x820	PROG_PTR_LSB	0: program_lsb_pointer	Contain the 32 lsb part of DDR address where the program start	0	32	0	RW
0x824	PROG_PTR_MSB	0: program_msb_pointer	Contain the 32 msb part of DDR address where the program start	0	32	0	RW
0x828	PROG_LENGTH	0: program_length	Size in Bytes of the program which store in the DDR	0	32	0	RW
0x82c	PROG_BATCH	0: program_batch_size	Number of times which the program is running and the input/output base address is incremented with the step size	0	32	1	RW
0x830	IP_STATUS_LSB	0: core_status_lsb	Engine Status Bits - lsb part	0	32	0	R
0x834	IP_STATUS_MSB	0: core_status_msb	Engine Status Bits - msb part	0	32	0	R
0x838	IP_PCOUNTER_LSB	0: perf_counter_lsb	Clock count 32 bits - lsb part	0	32	0	R
0x83c	IP_PCOUNTER_MSB	0: perf_counter_msb	Clock count 32 bits - msb part	0	32	0	R
0x840	TABLE_BASE_LSB	0: table_base_lab_addr	—	0	32	0	RW

Table 3-3. Additional Registers for Unstructured Compression (continued)

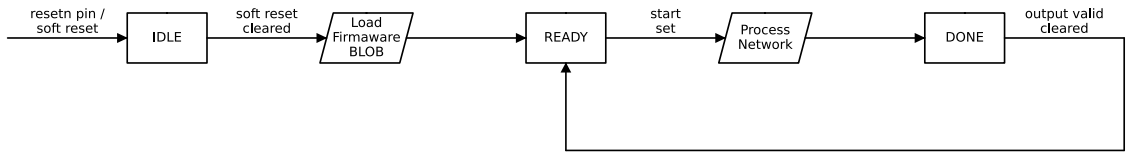
Register Address	Register Name	Description					Access
		Field_Name	Function	Start Bit	Num Bits	Default Value	
0x844	TABLE_BASE_MSB	0: table_base_msb_addr	—	0	32	0	RW
0x848	IN_BASE_LSB	0: input_base_lab_addr	—	0	32	0	RW
0x84c	IN_BASE_MSB	0: input_base_msb_addr	—	0	32	0	RW
0x850	OUT_BASE_LSB	0: output_base_lab_addr	—	0	32	0	RW
0x854	OUT_BASE_MSB	0: output_base_msb_addr	—	0	32	0	RW
0x858	IN_STEP_SIZE	0: input_step_size	—	0	32	0	RW
0x85c	OUT_STEP_SIZE	0: output_step_size	—	0	32	0	RW
0x860	INPUT_IMAGE_SIZE	0: input_image_width	input_image_size[15:0] - is the width of the image - how many bytes in lines; 512 pixels means 1536 bytes	0	16	1536	RW
		16: input_image_height	input_image_size[31:16] - is the height of the image - how many lines in the image	16	16	512	RW
0x880	IP_ECOUNTER_LSB	0: perf_engine_counter_lsb	Read the clock count for engine processing only - from first conv till end conv command - lsb part	0	32	0	RO
0x884	IP_ECOUNTER_LSB	0: perf_engine_counter_msb	Read the clock count for engine processing only - from first conv till end conv command - msb part	0	32	0	RO
0x890	MXP_BASE_LSB	0: mxp_base_lab_addr	—	0	32	0	RW
0x894	MXP_BASE_MSB	0: mxp_base_msb_addr	—	0	32	0	RW

Note: The listed registers are only available when unstructured compression is selected.

3.2. Network Processing (Ask a Question)

The following figure shows the flow of processing networks.

Figure 3-1. Network Processing Flow



The CoreVectorBlox SDK provides a C API.

When coming out of reset, CoreVectorBlox is held in an IDLE state.

Processing a network is started by setting the network BLOB address and then setting the 'start' bit of the control register. CoreVectorBlox will then begin parsing the network BLOB and read in the inputs as it starts processing. Exact steps of processing depend on the network BLOB. During processing, the memory master may also access temporary buffer memory. The temporary

buffers are pre-allocated inside the network BLOB; CoreVectorBlox only accesses memory inside the network BLOB and network input and output buffers.

When network processing completes, the 'Output Valid' bit of the control register is set and the network outputs can be read. The next network run might start as soon as the current run is finished.

At any point, CoreVectorBlox can be put back into the Soft Reset state by setting the 'soft reset' bit of the control register. Soft reset attempts to finish any outstanding AXI transactions on the master and slave interfaces, as opposed to a hard reset using the `resetn` pin, which will immediately cease all transactions and reset those interfaces. For more details on signals, see Memory Map.

4. Generics (Ask a Question)

Customers can define the generics listed in the following table as needed in the source code.

Table 4-1. CoreVectorBlox Generics

Generic	Default Setting	Valid Values	Description
Size Configuration	V1000	[V250, V500, V1000]	Size Configuration; see Processor Size Configuration Details table.
MX_AXI_DATA_WIDTH	256	[64, 128, 256]	AXI4 Data Master data width in bits.
Compression	Structured	[Disabled, Structured, Unstructured]	CNN Acceleration Method
AXI Ports	2	[1,2]	Number of AXI4 Data Master ports

Notes:

- Size, AXI ports and AXI_DATA_WIDTH are available only when no compression or compression is selected.
- For unstuctured compression, number of AXI masters is set to 2, and AXI_DATA_WIDTH is set to 256-bit for M0 port and 128-bit for M1 port.

5. Interface Description (Ask a Question)

The port signals for CoreVectorBlox are defined in the following tables and are also described in I/O Signal Diagram.

5.1. Clocks and Resets (Ask a Question)

The following table lists the clocks and the resets.

Table 5-1. Clocks and Resets

Signal	Function	I/O	Description
clk	Clock	Input	System clock. The control slave and data master are synchronous to this clock.
clk_2x	2X Clock	Input	Double-frequency clock. It is used as core clock for Vector Processor. It must be synchronous to and in phase with the clk. See the following notes.
clk_cnn	CNN Core Clock	Input	This is the asynchronous core processing clock for CNN accelerator.
resetn	Reset	Input	System reset (active-low). Resets all core functions as well as the control slave and data master.

Notes:

1. To minimize skew between clk and clk_2x, the clocks must be created from the same CCC on PolarFire devices. Additionally, the clock outputs are paired. Either the OUT0 and OUT1 or the OUT2 and OUT3 pair must be used, but not one output from each pair.
2. The Libero Place and Route tool must be configured with the “Repair Minimum Delay Violations” option selected to repair any hold violations caused by skew between the two clocks.

The following table lists the MAX clock frequencies.

Table 5-2. MAX Clock Frequencies

Configuration	clk	clk_2x	clk_cnn
V1000	130 MHz	260 MHz	260 MHz
V1000 Compression	125 MHz	250 MHz	250 MHz
Unstructured Compression	150 MHz	300 MHz	200 MHz

Notes:

- Max frequencies provided are for V1000 Size Configuration. Actual clock frequency can very based on the size configuration and device utilization.
- When using the no compression or compression configuration, clk_cnn can be tied to clk_2x.

5.2. Control Slave Signals (Ask a Question)

The Control Slave is an AXI4-Lite compliant interface with the memory map described in the Memory Map section. It is synchronous to clk and reset by `resetn` pin. The following table lists the signals.

Table 5-3. Control Slave Signals

Signal	Function	I/O	Description
s_axi_awaddr	Write Address	Input	AXI4-Lite slave write address.
s_axi_awvalid	Write Address Valid	Input	AXI4-Lite slave write address valid.
s_axi_awready	Write Address Ready	Output	AXI4-Lite slave write address ready.
s_axi_wdata	Write Data	Input	AXI4-Lite slave write data.

Table 5-3. Control Slave Signals (continued)

Signal	Function	I/O	Description
s_axi_wstrb	Write Data Strobe	Input	AXI4-Lite slave write data strobe (byte enable). CoreVectorBlox expects all write strobe signals to be all high or all low; partial register writes results in undefined results.
s_axi_wvalid	Write Data Valid	Input	AXI4-Lite slave write data valid.
s_axi_wready	Write Data Ready	Output	AXI4-Lite slave write data ready.
s_axi_bready	Write Response Ready	Input	AXI4-Lite slave write response ready.
s_axi_bresp	Write Response	Output	AXI4-Lite slave write response code.
s_axi_bvalid	Write Response Valid	Output	AXI4-Lite slave write response valid.
s_axi_araddr	Read Address	Input	AXI4-Lite slave read address.
s_axi_arvalid	Read Address Valid	Input	AXI4-Lite slave read address valid.
s_axi_arready	Read Address Ready	Output	AXI4-Lite slave read address ready.
s_axi_rready	Read Data Ready	Input	AXI4-Lite slave read data ready.
s_axi_rdata	Read Data	Output	AXI4-Lite slave read data.
s_axi_rresp	Read Data Response	Output	AXI4-Lite slave read data response code.
s_axi_rvalid	Read Data Valid	Output	AXI4-Lite slave read data valid.

Note:

1. All control slave signals are synchronous to clk.
2. The control slave is reset by `resetn` pin.

5.3. Data Master Signals (Ask a Question)

Two Data Master ports are an AXI4 compliant interface. It is synchronous to clk and reset by `resetn` pin. The following table lists the signals.

Table 5-4. Data Master Signals

Signal	Function	I/O	Description
mx_axi_arready	Read Address Ready	Input	AXI4 master read address ready.
mx_axi_arvalid	Read Address Valid	Output	AXI4 master read address valid.
mx_axi_arid	Read Address ID	Output	AXI4 master read address ID. CoreVectorBlox uses multiple IDs; these must be propagated correctly through any interconnect attached to the Data Master.
mx_axi_araddr	Read Address	Output	AXI4 master read address.
mx_axi_arlen	Read Length	Output	AXI4 master read length (beats per burst minus 1).
mx_axi_arsize	Read Size	Output	AXI4 master read size. CoreVectorBlox does not issue narrow reads; this will be fixed to the data width size.
mx_axi_arburst	Read Burst Type	Output	AXI4 master read burst type. CoreVectorBlox only issues incrementing bursts.
mx_axi_arprot	Read Protection	Output	AXI4 master read protection. CoreVectorBlox only issues unprivileged, secure data acceses.

Table 5-4. Data Master Signals (continued)

Signal	Function	I/O	Description
mx_axi_arcache	Read Transaction Attributes	Output	AXI4 master read transaction attributes. CoreVectorBlox issues modifiable and bufferable transactions. It does not set allocation bits and assumes that transactions can be read from memory in systems with caches.
mx_axi_rready	Read Data Ready	Output	AXI4 master read data ready.
mx_axi_rvalid	Read Data Valid	Input	AXI4 master read data.
mx_axi_rid	Read Data ID	Input	AXI4 master read data ID. CoreVectorBlox uses multiple IDs.
mx_axi_rdata	Read Data	Input	AXI4 master read data.
mx_axi_rresp	Read Data Response	Input	AXI4 master read data response code.
mx_axi_rlast	Read Data Last	Input	AXI4 master read data last (end of burst).
mx_axi_awready	Write Address Ready	Input	AXI4 master write address ready.
mx_axi_awvalid	Write Address Valid	Output	AXI4 master write address valid.
mx_axi_awid	Write Address ID	Output	AXI4 master write address ID. CoreVectorBlox uses multiple IDs; these must be propagated correctly through any interconnect attached to the Data Master.
mx_axi_awaddr	Write Address	Output	AXI4 master write address.
mx_axi_awlen	Write Length	Output	AXI4 master write length (beats per burst minus 1).
mx_axi_awsize	Write Size	Output	AXI4 master write size. CoreVectorBlox does not issue narrow writes; this will be fixed to the data width size.
mx_axi_awburst	Write Burst Type	Output	AXI4 master write burst type. CoreVectorBlox only issues incrementing bursts.
mx_axi_awprot	Write Protection	Output	AXI4 master write protection. CoreVectorBlox only issues unprivileged, secure data acceses.
mx_axi_awcache	Write Transaction Attributes	Output	AXI4 master write transaction attributes. CoreVectorBlox issues modifiable, bufferable transactions. It does not set allocation bits.
mx_axi_wready	Write Data Ready	Input	AXI4 master write data ready.
mx_axi_wvalid	Write Data Valid	Output	AXI4 master write data valid.
mx_axi_wdata	Write Data	Output	AXI4 master write data.
mx_axi_wstrb	Write Data Strobe	Output	AXI4 master write data strobe (byte enable).
mx_axi_wlast	Write Data Last	Output	AXI4 master write last (end of burst).
mx_axi_bready	Write Response Ready	Output	AXI4 master write response ready.
mx_axi_bvalid	Write Response Valid	Input	AXI4 master write response valid.
mx_axi_bid	Write Response ID	Input	AXI4 master write response ID.
mx_axi_bresp	Write Response Code	Input	AXI4 master write response code.

Notes:

1. All data master signals are synchronous to clk.
2. The data master is reset by `resetn` pin.
3. All ID ID signals have been increased to 4 bits.

5.4. Interrupt Signals (Ask a Question)

The following table lists the interrupt signals.

Table 5-5. Interrupt Signals

Signal	Function	I/O	Description
output_valid	Interrupt Output	Output	Mirrors the Output Valid bit in the Control Register (see Memory Map) for use as an interrupt for needed systems.

6. Tool Flows (Ask a Question)

The following section shows the tool flows of CoreVectorBlox.

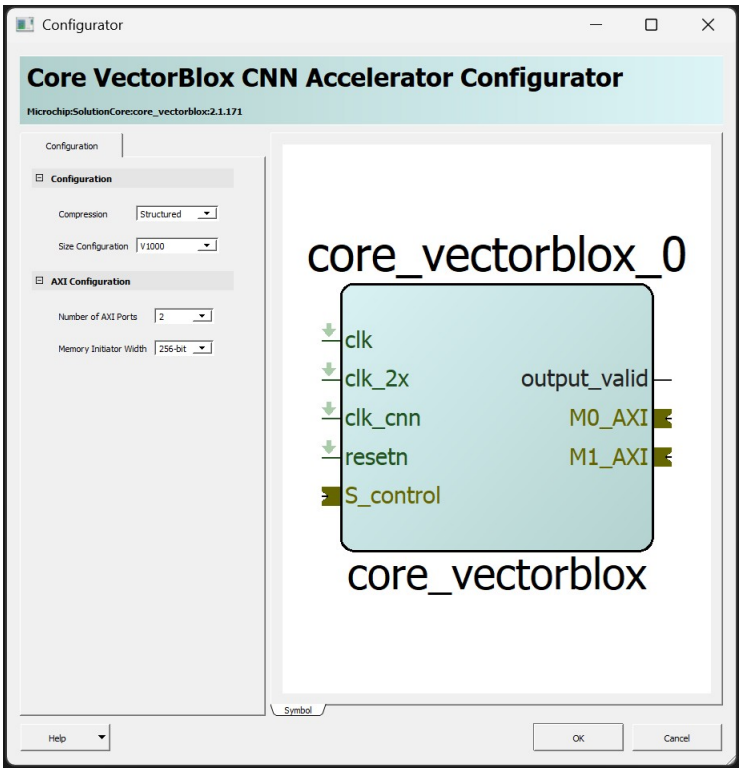
6.1. Licenses (Ask a Question)

CoreVectorBlox is licensed using encrypted RTL.

6.2. Smart Design (Ask a Question)

The following figure shows the core configured using the configuration GUI within SmartDesign.

Figure 6-1. CoreVectorBlox Configurator within SmartDesign with V1000 Configuration



6.3. Simulation (Ask a Question)

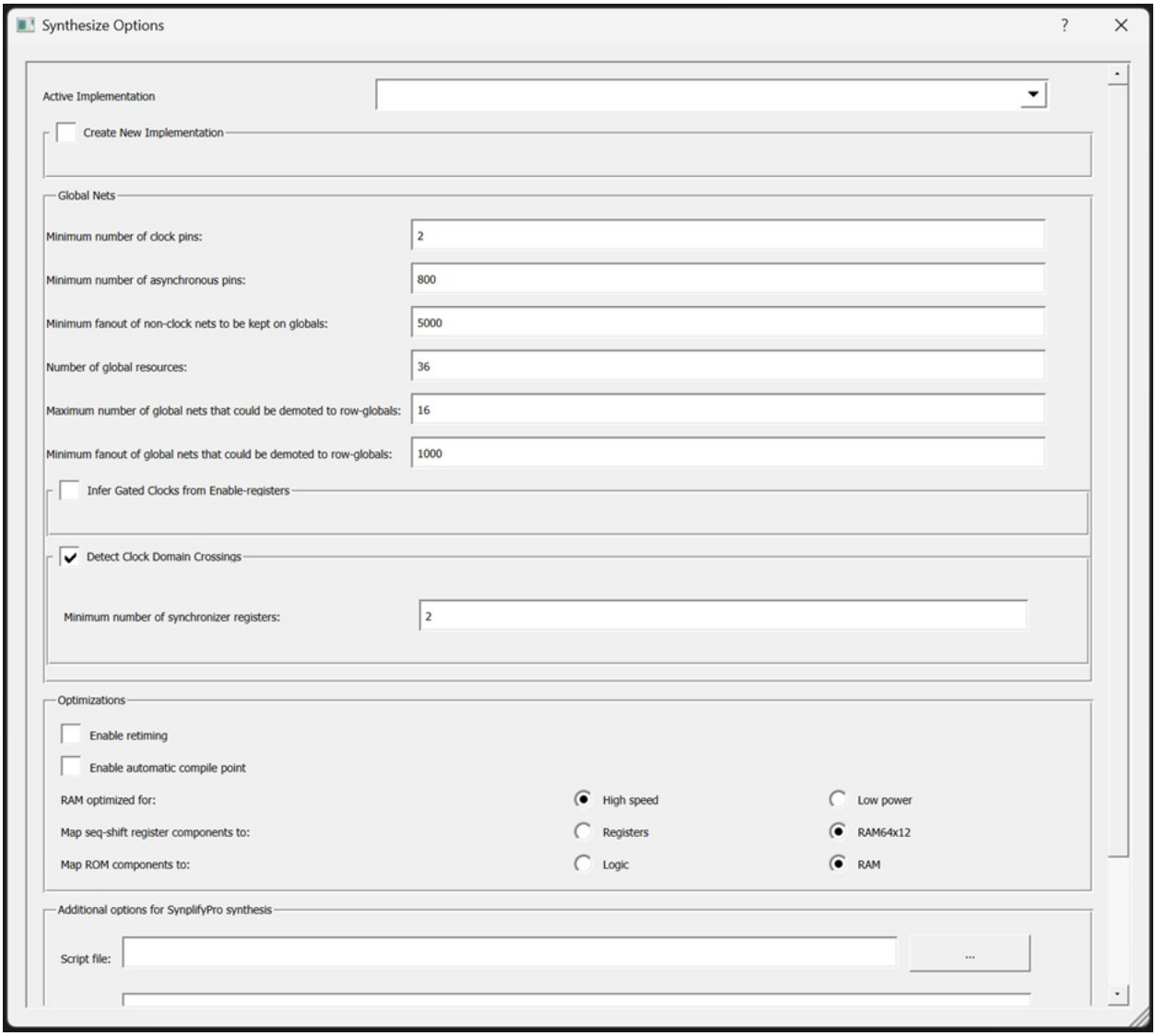
CoreVectorBlox can be functionally simulated. For functional simulation, see the SDK. Functional simulation is the preferred way of verifying network functionality and accuracy.

6.4. Synthesis (Ask a Question)

Set the design root appropriately and click the Synthesis icon in Libero. To perform synthesis, right-click and select Run. CoreVectorBlox requires no special synthesis settings.

When using the unstructured compression configuration “Enable automatic compile point” option must be unchecked and “Map ROM components to: “ must be set to RAM. The following figure shows the **Synthesis Options** window.

Figure 6-2. Synthesis Options



6.5. Place and Route (Ask a Question)

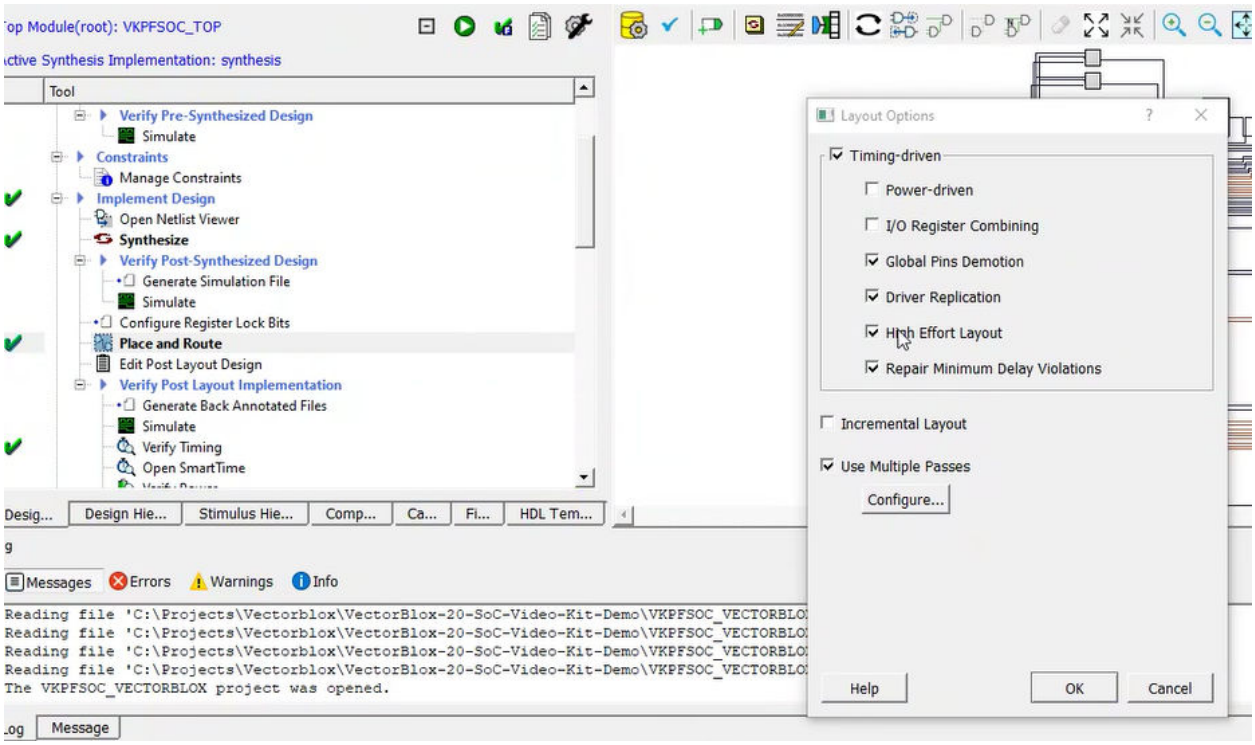
The “Repair Minimum Delay Violations” option must be turned on in the Libero **Place and Route** tool to fix any hold violations between clk and clk_2x caused by clock skew through the fabric (see Clocks and Resets).

Set the design route appropriately and run Synthesis. Click the **Place and Route** icon in Libero to invoke the Designer software.

For the best results, make the following settings in the **Place and Route** options in Libero.

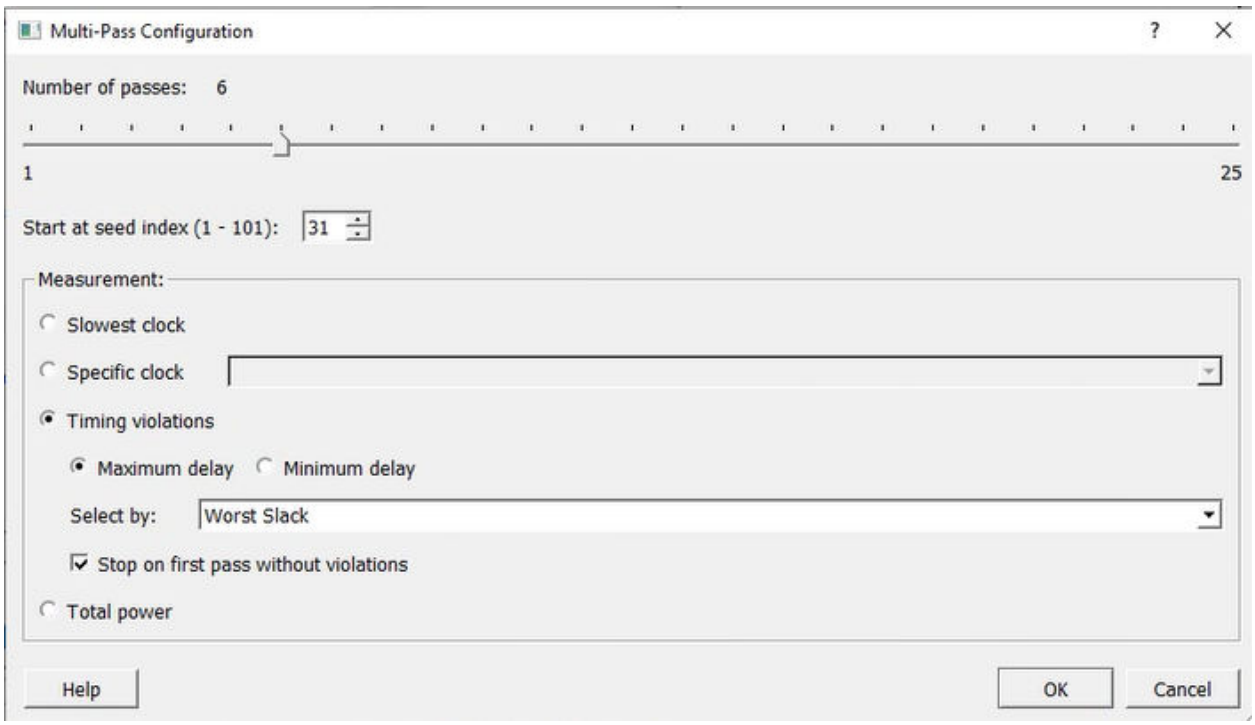
The following figure shows the Place and Route settings.

Figure 6-3. Place and Route Settings



The following figure shows the Multi-Pass configuration settings.

Figure 6-4. Multi-Pass Configuration Settings



7. Revision History (Ask a Question)

The revision history describes the changes that were implemented in the document. The changes are listed by revision, starting with the current publication.

Revision	Date	Description
D	05/2026	Updated the title of the document to “CoreVectorBlox IP Handbook” from “CoreVectorBlox v3.0 IP Handbook”.
C	12/2025	The following is the list of changes made in revision C: • Updated “CoreVectorBlox v3.0” from “CoreVectorBlox v2.0” in Core Versions section. • Updated Figure 1-1 in Overview section. • Updated Clocks and Resets section. • Removed “Processor Size Configuration Details” table and updated Configuration Options section. • Updated Data Master Signals section. • Updated Features section. • Removed “Error Codes” table updated Memory Components section. • Updated Figure 2-2 in Hardware Architecture section. • Added Table 3-3 and Note in Memory Map section. • Updated Table 4-1 in Generics section.
B	11/2024	The following is the list of changes in revision B: • Updated “CoreVectorBlox v2.0” from “CoreVectorBlox v1.1” in Core Versions section. • Updated Supported Families section. • Removed "1.4 Device Utilization and Performance" section. • Updated Memory Components section. • “Controller” is added and updated Hardware Architecture section. • Updated “Processor Size Configuration Details” table in Configuration Options section. • Updated Table 3-1 and “Error Codes” table in Memory Map section. • Updated Network Processingsection. • Updated Table 4-1 in Generics section. • Updated Simulation section. • Added Figure 6-3 and Figure 6-4 in Place and Route sections.
A	01/2021	The following is the list of changes in revision A. • The document was updated to Microchip template and document number was changed from 50200919 to DS50003112A.
2.0	11/2020	The following is the list of changes in revision 2.0. • The Overview section was updated. • “Device Utilization and Performance” table was updated. • Added Interrupt Signals section. • Updated Figure 6-1.
1.0	06/2020	Revision 1.0 is the first publication of this document.

Microchip FPGA Support

Microchip FPGA products group backs its products with various support services, including Customer Service, Customer Technical Support Center, a website, and worldwide sales offices. Customers are suggested to visit Microchip online resources prior to contacting support as it is very likely that their queries have been already answered.

Contact Technical Support Center through the website at www.microchip.com/support. Mention the FPGA Device Part number, select appropriate case category, and upload design files while creating a technical support case.

Contact Customer Service for non-technical product support, such as product pricing, product upgrades, update information, order status, and authorization.

- From North America, call **800.262.1060**
- From the rest of the world, call **650.318.4460**
- Fax, from anywhere in the world, **650.318.8044**

Microchip Information

Trademarks

The “Microchip” name and logo, the “M” logo, and other names, logos, and brands are registered and unregistered trademarks of Microchip Technology Incorporated or its affiliates and/or subsidiaries in the United States and/or other countries (“Microchip Trademarks”). Information regarding Microchip Trademarks can be found at https://www.microchip.com/en-us/about/legal-information/microchip-trademarks.

ISBN: 979-8-3371-3319-5

Legal Notice

This publication and the information herein may be used only with Microchip products, including to design, test, and integrate Microchip products with your application. Use of this information in any other manner violates these terms. Information regarding device applications is provided only for your convenience and may be superseded by updates. It is your responsibility to ensure that your application meets with your specifications. Contact your local Microchip sales office for additional support or, obtain additional support at www.microchip.com/en-us/support/design-help/client-support-services.

THIS INFORMATION IS PROVIDED BY MICROCHIP “AS IS”. MICROCHIP MAKES NO REPRESENTATIONS OR WARRANTIES OF ANY KIND WHETHER EXPRESS OR IMPLIED, WRITTEN OR ORAL, STATUTORY OR OTHERWISE, RELATED TO THE INFORMATION INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A PARTICULAR PURPOSE, OR WARRANTIES RELATED TO ITS CONDITION, QUALITY, OR PERFORMANCE.

IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE, INCIDENTAL, OR CONSEQUENTIAL LOSS, DAMAGE, COST, OR EXPENSE OF ANY KIND WHATSOEVER RELATED TO THE INFORMATION OR ITS USE, HOWEVER CAUSED, EVEN IF MICROCHIP HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP’S TOTAL LIABILITY ON ALL CLAIMS IN ANY WAY RELATED TO THE INFORMATION OR ITS USE WILL NOT EXCEED THE AMOUNT OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THE INFORMATION.

Use of Microchip devices in life support and/or safety applications is entirely at the buyer’s risk, and the buyer agrees to defend, indemnify and hold harmless Microchip from any and all damages, claims, suits, or expenses resulting from such use. No licenses are conveyed, implicitly or otherwise, under any Microchip intellectual property rights unless otherwise stated.

Microchip Devices Code Protection Feature

Note the following details of the code protection feature on Microchip products:

- Microchip products meet the specifications contained in their particular Microchip Data Sheet.
- Microchip believes that its family of products is secure when used in the intended manner, within operating specifications, and under normal conditions.
- Microchip values and aggressively protects its intellectual property rights. Attempts to breach the code protection features of Microchip products are strictly prohibited and may violate the Digital Millennium Copyright Act.
- Neither Microchip nor any other semiconductor manufacturer can guarantee the security of its code. Code protection does not mean that we are guaranteeing the product is “unbreakable”. Code protection is constantly evolving. Microchip is committed to continuously improving the code protection features of our products.